nLP2

December 17, 2025

[1]:
```
# ============================================================
# CELL 1: SETUP, DATA PREP & EXPERT LABEL GENERATION
# ============================================================
print("  SUBTASK 2: DUAL-MODEL EXPERT STRATEGY")
print("=" * 60)

# 1. Install Dependencies (Quietly)
!pip install -q torch transformers[torch] scikit-learn pandas gdown accelerate␣
 ↪nltk

import os
import torch
import pandas as pd
import numpy as np
import gdown
import gc
import json
import warnings
import logging
from sklearn.model_selection import train_test_split
from datetime import datetime

# 2. Configuration & Silence Logs
warnings.filterwarnings('ignore')
os.environ["WANDB_DISABLED"] = "true"
os.environ["TRANSFORMERS_NO_ADVISORY_WARNINGS"] = "true"
logging.getLogger("transformers").setLevel(logging.ERROR)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"  Device: {DEVICE}")

SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

# 3. Download Data
files = {
```

```
    'data_eng.csv': '190j4NYhCFyf3wJgsLpPGolm45K-9Idx2',
    'test_eng.csv': '1cMxtHb_bZH1NY19rr7ROizXIzf5xj6Va',
    'data_swa.csv': '1-Mk-PFY2r9PSxac7KJENPq0EcAlKJaij',
    'test_swa.csv': '1dtROGr4go8FFHSMlmYC31brgooUU5ixX',
}

print("\n Downloading Data...")
for name, file_id in files.items():
    if not os.path.exists(name):
        gdown.download(f'https://drive.google.com/uc?id={file_id}', name,␣
 ↪quiet=True)

# 4. Load & Clean Data
def load_clean(filename):
    try:
        df = pd.read_csv(filename)
        if 'text' in df.columns:
            df['text'] = df['text'].astype(str).str.strip()
            df = df[df['text'].str.len() > 0]
        return df
    except: return pd.DataFrame()

df_eng = load_clean('data_eng.csv')
df_swa = load_clean('data_swa.csv')
test_eng = load_clean('test_eng.csv')
test_swa = load_clean('test_swa.csv')

# 5. Filter Polarized Rows Only (Subtask 2 Requirement)
def get_polarized(df):
    col = next((c for c in df.columns if 'polar' in c.lower()), None)
    if col: return df[df[col] == 1].copy()
    return df

df_eng_polar = get_polarized(df_eng)
df_swa_polar = get_polarized(df_swa)

print(f" Training Data (Polarized Only): English={len(df_eng_polar)},␣
 ↪Swahili={len(df_swa_polar)}")

# 6. Expert Label Generation (Multilingual)
POLARIZATION_TYPES = ['political', 'racial_ethnic', 'religious',␣
 ↪'gender_sexual', 'other']

# Updated to include SWAHILI keywords so the Swahili model learns correctly
KEYWORD_SYSTEMS = {
```

```
    'political': ['politic', 'government', 'elect', 'vote', 'party',␣
 ↪'president', 'minister', 'policy', 'law', 'siasa', 'serikali', 'kura',␣
 ↪'chama', 'rais', 'waziri', 'sheria', 'bunge'],
    'racial_ethnic': ['race', 'racial', 'ethnic', 'black', 'white', 'asian',␣
 ↪'tribe', 'tribal', 'xenophob', 'kabila', 'ukabila', 'rangi', 'mzungu',␣
 ↪'mwafrika', 'ubaguzi'],
    'religious': ['religion', 'god', 'allah', 'jesus', 'church', 'mosque',␣
 ↪'muslim', 'christian', 'dini', 'mungu', 'kanisa', 'msikiti', 'uislamu',␣
 ↪'ukristo'],
    'gender_sexual': ['gender', 'sex', 'lgbt', 'gay', 'woman', 'man',␣
 ↪'feminist', 'jinsia', 'mwanamke', 'mwanaume', 'mapenzi'],
    'other': ['sport', 'football', 'tech', 'money', 'business', 'climate',␣
 ↪'michezo', 'mpira', 'pesa', 'biashara', 'uchumi']
}

def generate_labels(texts):
    labels = np.zeros((len(texts), len(POLARIZATION_TYPES)), dtype=int)
    for i, text in enumerate(texts):
        text_lower = str(text).lower()
        scores = [0] * 5
        for idx, cat in enumerate(POLARIZATION_TYPES):
            # Check combined list of Eng/Swa keywords
            for kw in KEYWORD_SYSTEMS.get(cat, []):
                if kw in text_lower: scores[idx] += 1

        if sum(scores) > 0: labels[i, np.argmax(scores)] = 1
        else: labels[i, 0] = 1 # Default to Political
    return labels

print("\n Generating Synthetic Labels (English + Swahili)...")
eng_labels = generate_labels(df_eng_polar['text'].tolist())
swa_labels = generate_labels(df_swa_polar['text'].tolist())

# 7. Train/Val Split
train_eng_txt, val_eng_txt, train_eng_lbl, val_eng_lbl = train_test_split(
    df_eng_polar['text'].tolist(), eng_labels, test_size=0.2, random_state=SEED
)
train_swa_txt, val_swa_txt, train_swa_lbl, val_swa_lbl = train_test_split(
    df_swa_polar['text'].tolist(), swa_labels, test_size=0.2, random_state=SEED
)

print(" Data Preparation Complete")
```

```
 SUBTASK 2: MULTI-LABEL SOURCE IDENTIFICATION
============================================================
 Data Ready. English Train: 2738, Swahili Train: 5942
```

[1]:

[1]:

[2]:
```
# ============================================================
# CELL 2: TRAINING WITH CLASS WEIGHTS
# ============================================================
print("\n TRAINING: With class weights for imbalance...")

from transformers import AutoTokenizer, AutoModelForSequenceClassification,␣
 ↪TrainingArguments, Trainer
import torch.nn as nn

# Add class weights to handle imbalance
def compute_class_weights(labels):
    weights = []
    n_samples = labels.shape[0]
    for i in range(labels.shape[1]):
        pos = labels[:, i].sum()
        neg = n_samples - pos
        if pos > 0:
            weights.append(neg / pos)
        else:
            weights.append(1.0)
    return torch.tensor(weights, dtype=torch.float).to(DEVICE)

# Compute weights
eng_weights = compute_class_weights(eng_tr_lbl)
swa_weights = compute_class_weights(swa_tr_lbl)
print(f"English class weights: {eng_weights.cpu().numpy()}")
print(f"Swahili class weights: {swa_weights.cpu().numpy()}")

# Model with weighted loss
class WeightedMultiLabelModel(nn.Module):
    def __init__(self, model_name, class_weights):
        super().__init__()
        self.model = AutoModelForSequenceClassification.from_pretrained(
            model_name,
            num_labels=len(TARGET_COLS),
            problem_type="multi_label_classification"
        )
        self.loss_fct = nn.BCEWithLogitsLoss(pos_weight=class_weights)

    def forward(self, input_ids, attention_mask, labels=None):
        outputs = self.model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        loss = None
```

```
        if labels is not None:
            loss = self.loss_fct(logits, labels)
        return {'loss': loss, 'logits': logits}

# Dataset
class SimpleDataset(torch.utils.data.Dataset):
    def __init__(self, texts, labels, tokenizer):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            str(self.texts[idx]),
            truncation=True,
            padding='max_length',
            max_length=128,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(self.labels[idx], dtype=torch.float)
        }

# Train English model
print("\n  Training English model (RoBERTa)...")
eng_tokenizer = AutoTokenizer.from_pretrained("roberta-base")
eng_model = WeightedMultiLabelModel("roberta-base", eng_weights).to(DEVICE)

eng_trainer = Trainer(
    model=eng_model,
    args=TrainingArguments(
        output_dir="./eng_model",
        num_train_epochs=3,
        per_device_train_batch_size=16,
        learning_rate=2e-5,
        save_strategy="no",
        report_to="none",
        remove_unused_columns=False
    ),
    train_dataset=SimpleDataset(eng_tr_txt, eng_tr_lbl, eng_tokenizer),
    eval_dataset=SimpleDataset(eng_val_txt, eng_val_lbl, eng_tokenizer),
)
```

```
eng_trainer.train()
print("  English model trained")

# Train Swahili model
print("\n  Training Swahili model (Afro-XLMR)...")
swa_tokenizer = AutoTokenizer.from_pretrained("Davlan/afro-xlmr-base")
swa_model = WeightedMultiLabelModel("Davlan/afro-xlmr-base", swa_weights).
 ↪to(DEVICE)

swa_trainer = Trainer(
    model=swa_model,
    args=TrainingArguments(
        output_dir="./swa_model",
        num_train_epochs=3,
        per_device_train_batch_size=16,
        learning_rate=2e-5,
        save_strategy="no",
        report_to="none",
        remove_unused_columns=False
    ),
    train_dataset=SimpleDataset(swa_tr_txt, swa_tr_lbl, swa_tokenizer),
    eval_dataset=SimpleDataset(swa_val_txt, swa_val_lbl, swa_tokenizer),
)
swa_trainer.train()
print("  Swahili model trained")
```

```
 STARTING 6-MODEL EXPERT TRAINING…

============================================================
 TRAIN START: English | Model: microsoft/deberta-v3-base
============================================================

tokenizer_config.json:   0%|          | 0.00/52.0 [00:00<?, ?B/s]

config.json:   0%|          | 0.00/579 [00:00<?, ?B/s]

spm.model:   0%|          | 0.00/2.46M [00:00<?, ?B/s]

pytorch_model.bin:   0%|          | 0.00/371M [00:00<?, ?B/s]

Some weights of DebertaV2ForSequenceClassification were not initialized from the
model checkpoint at microsoft/deberta-v3-base and are newly initialized:
['classifier.bias', 'classifier.weight', 'pooler.dense.bias',
'pooler.dense.weight']
You should probably TRAIN this model on a down-stream task to be able to use it
for predictions and inference.

model.safetensors:   0%|          | 0.00/371M [00:00<?, ?B/s]

<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>

  Saved: ./model_eng_1 (F1: 0.8311)

============================================================
  TRAIN START: English | Model: roberta-base
============================================================

tokenizer_config.json:   0%|          | 0.00/25.0 [00:00<?, ?B/s]

config.json:   0%|          | 0.00/481 [00:00<?, ?B/s]

vocab.json:   0%|          | 0.00/899k [00:00<?, ?B/s]

merges.txt:   0%|          | 0.00/456k [00:00<?, ?B/s]

tokenizer.json:   0%|          | 0.00/1.36M [00:00<?, ?B/s]

model.safetensors:   0%|          | 0.00/499M [00:00<?, ?B/s]

Some weights of RobertaForSequenceClassification were not initialized from the
model checkpoint at roberta-base and are newly initialized:
['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias',
'classifier.out_proj.weight']
You should probably TRAIN this model on a down-stream task to be able to use it
for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

  Saved: ./model_eng_2 (F1: 0.8855)

============================================================
  TRAIN START: English | Model: bert-base-uncased
============================================================

tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]

config.json:   0%|          | 0.00/570 [00:00<?, ?B/s]

vocab.txt:   0%|          | 0.00/232k [00:00<?, ?B/s]

tokenizer.json:   0%|          | 0.00/466k [00:00<?, ?B/s]

model.safetensors:   0%|          | 0.00/440M [00:00<?, ?B/s]

Some weights of BertForSequenceClassification were not initialized from the
model checkpoint at bert-base-uncased and are newly initialized:
['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it
for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>
```

```
  Saved: ./model_eng_3 (F1: 0.9035)

============================================================
  TRAIN START: Swahili | Model: Davlan/afro-xlmr-base
============================================================

tokenizer_config.json:   0%|          | 0.00/398 [00:00<?, ?B/s]

sentencepiece.bpe.model:   0%|          | 0.00/5.07M [00:00<?, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

special_tokens_map.json:   0%|          | 0.00/239 [00:00<?, ?B/s]

config.json:   0%|          | 0.00/707 [00:00<?, ?B/s]

model.safetensors:   0%|          | 0.00/1.11G [00:00<?, ?B/s]

Some weights of XLMRobertaForSequenceClassification were not initialized from
the model checkpoint at Davlan/afro-xlmr-base and are newly initialized:
['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias',
'classifier.out_proj.weight']
You should probably TRAIN this model on a down-stream task to be able to use it
for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

  Saved: ./model_swa_1 (F1: 0.8995)

============================================================
  TRAIN START: Swahili | Model: xlm-roberta-base
============================================================

tokenizer_config.json:   0%|          | 0.00/25.0 [00:00<?, ?B/s]

config.json:   0%|          | 0.00/615 [00:00<?, ?B/s]

sentencepiece.bpe.model:   0%|          | 0.00/5.07M [00:00<?, ?B/s]

tokenizer.json:   0%|          | 0.00/9.10M [00:00<?, ?B/s]

model.safetensors:   0%|          | 0.00/1.12G [00:00<?, ?B/s]

Some weights of XLMRobertaForSequenceClassification were not initialized from
the model checkpoint at xlm-roberta-base and are newly initialized:
['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias',
'classifier.out_proj.weight']
You should probably TRAIN this model on a down-stream task to be able to use it
for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>
```

```
  Saved: ./model_swa_2 (F1: 0.8585)

============================================================
  TRAIN START: Swahili | Model: bert-base-multilingual-cased
============================================================

tokenizer_config.json:   0%|          | 0.00/49.0 [00:00<?, ?B/s]

config.json:   0%|          | 0.00/625 [00:00<?, ?B/s]

vocab.txt:   0%|          | 0.00/996k [00:00<?, ?B/s]

tokenizer.json:   0%|          | 0.00/1.96M [00:00<?, ?B/s]

model.safetensors:   0%|          | 0.00/714M [00:00<?, ?B/s]

Some weights of BertForSequenceClassification were not initialized from the
model checkpoint at bert-base-multilingual-cased and are newly initialized:
['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it
for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

  Saved: ./model_swa_3 (F1: 0.8807)

============================================================
  SUBTASK 2 MODEL LEADERBOARD
============================================================
| Language   | Model                          |   F1 Macro |
|:-----------|:-------------------------------|-----------:|
| English    | bert-base-uncased              |   0.903485 |
| English    | roberta-base                   |   0.885454 |
| English    | microsoft/deberta-v3-base      |   0.831063 |
| Swahili    | Davlan/afro-xlmr-base          |   0.899539 |
| Swahili    | bert-base-multilingual-cased   |   0.880724 |
| Swahili    | xlm-roberta-base               |   0.858501 |
============================================================
```

[2]:

[3]:
```python
# ============================================================
# CELL 3: PREDICTION & SUBMISSION (STRICT FORMAT)
# ============================================================
print("\n GENERATING PREDICTIONS (6-MODEL ENSEMBLE)...")

from tqdm.auto import tqdm
import torch.nn.functional as F

def predict_ensemble(texts, model_paths):
```

```
    if len(texts) == 0: return []
    ensemble_probs = None
    valid_models = 0

    for path in model_paths:
        if not os.path.exists(path): continue
        print(f"  -> Model: {path}...")

        tokenizer = AutoTokenizer.from_pretrained(path)
        model = AutoModelForSequenceClassification.from_pretrained(path).
 ↪to(DEVICE)
        model.eval()

        batch_probs = []
        for i in tqdm(range(0, len(texts), 64), leave=False):
            batch = texts[i:i+64]
            inputs = tokenizer(batch, padding=True, truncation=True,␣
 ↪max_length=128, return_tensors="pt").to(DEVICE)
            with torch.no_grad():
                outputs = model(**inputs)
                # Sigmoid for multi-label
                probs = torch.sigmoid(outputs.logits).cpu().numpy()
                batch_probs.extend(probs)

        batch_probs = np.array(batch_probs)
        if ensemble_probs is None: ensemble_probs = batch_probs
        else: ensemble_probs += batch_probs
        valid_models += 1

    if valid_models == 0: return np.zeros((len(texts), 5))
    # Average and Threshold
    return (ensemble_probs / valid_models > 0.5).astype(int)

# Load Test
test_eng = pd.read_csv('test_eng.csv')
test_swa = pd.read_csv('test_swa.csv')
test_eng['text'] = test_eng['text'].astype(str).str.strip()
test_swa['text'] = test_swa['text'].astype(str).str.strip()

# Predict
print("\n  Predicting English...")
eng_preds = predict_ensemble(test_eng['text'].tolist(), ["./model_eng_1", "./
 ↪model_eng_2", "./model_eng_3"])

print("\n  Predicting Swahili...")
swa_preds = predict_ensemble(test_swa['text'].tolist(), ["./model_swa_1", "./
 ↪model_swa_2", "./model_swa_3"])
```

```python
# ============================================================
# PART A: DEBUG FILES (WITH TEXT)
# ============================================================
debug_folder = "debug_subtask2_with_text"
if os.path.exists(debug_folder): shutil.rmtree(debug_folder)
os.makedirs(debug_folder)

cols = ['political', 'racial_ethnic', 'religious', 'gender_sexual', 'other']

# Save English Debug
eng_debug = test_eng.copy()
eng_debug[cols] = eng_preds
eng_debug.to_csv(f"{debug_folder}/eng_debug.csv", index=False)

# Save Swahili Debug
swa_debug = test_swa.copy()
swa_debug[cols] = swa_preds
swa_debug.to_csv(f"{debug_folder}/swa_debug.csv", index=False)

print(f"\n Debug files created in '{debug_folder}' (Use these to inspect␣
 ↪text)")


# ============================================================
# PART B: OFFICIAL CODABENCH ZIP (NO TEXT)
# ============================================================
submission_folder = "subtask_2"
if os.path.exists(submission_folder): shutil.rmtree(submission_folder)
os.makedirs(submission_folder)

# English Final (ID + 5 Columns ONLY)
eng_final = eng_debug[['id'] + cols]
eng_final.to_csv(f"{submission_folder}/pred_eng.csv", index=False)

# Swahili Final (ID + 5 Columns ONLY)
swa_final = swa_debug[['id'] + cols]
swa_final.to_csv(f"{submission_folder}/pred_swa.csv", index=False)

# Zip
from datetime import datetime
timestamp = datetime.now().strftime('%H%M%S')
zip_filename = f"submission_task2_{timestamp}"
shutil.make_archive(zip_filename, 'zip', root_dir='.',␣
 ↪base_dir=submission_folder)
```

```
print("\n" + "="*60)
print(f"  SUBMISSION READY FOR CODABENCH!")
print(f"  Download: {zip_filename}.zip")
print(f"    Structure: subtask_2/pred_eng.csv & pred_swa.csv")
print(f"    (Verified: No text columns, Correct folder name)")
print("="*60)
```

```
  GENERATING PREDICTIONS (6-MODEL ENSEMBLE)…

   Predicting English…
   -> Model: ./model_eng_1…
The tokenizer you are loading from './model_eng_1' with an incorrect regex
pattern: https://huggingface.co/mistralai/Mistral-
Small-3.1-24B-Instruct-2503/discussions/84#69121093e8b480e709447d5e. This will
lead to incorrect tokenization. You should set the `fix_mistral_regex=True` flag
when loading this tokenizer to fix this issue.
   0%|          | 0/3 [00:00<?, ?it/s]
   -> Model: ./model_eng_2…
   0%|          | 0/3 [00:00<?, ?it/s]
   -> Model: ./model_eng_3…
   0%|          | 0/3 [00:00<?, ?it/s]

   Predicting Swahili…
   -> Model: ./model_swa_1…
The tokenizer you are loading from './model_swa_1' with an incorrect regex
pattern: https://huggingface.co/mistralai/Mistral-
Small-3.1-24B-Instruct-2503/discussions/84#69121093e8b480e709447d5e. This will
lead to incorrect tokenization. You should set the `fix_mistral_regex=True` flag
when loading this tokenizer to fix this issue.
   0%|          | 0/6 [00:00<?, ?it/s]
   -> Model: ./model_swa_2…
The tokenizer you are loading from './model_swa_2' with an incorrect regex
pattern: https://huggingface.co/mistralai/Mistral-
Small-3.1-24B-Instruct-2503/discussions/84#69121093e8b480e709447d5e. This will
lead to incorrect tokenization. You should set the `fix_mistral_regex=True` flag
when loading this tokenizer to fix this issue.
   0%|          | 0/6 [00:00<?, ?it/s]
The tokenizer you are loading from './model_swa_3' with an incorrect regex
pattern: https://huggingface.co/mistralai/Mistral-
Small-3.1-24B-Instruct-2503/discussions/84#69121093e8b480e709447d5e. This will
```

```
lead to incorrect tokenization. You should set the `fix_mistral_regex=True` flag
when loading this tokenizer to fix this issue.
  -> Model: ./model_swa_3…
  0%|          | 0/6 [00:00<?, ?it/s]

  Debug files created in 'debug_subtask2_with_text' (Use these to inspect text)

============================================================
  SUBMISSION READY FOR CODABENCH!
  Download: submission_task2_095649.zip
    Structure: subtask_2/pred_eng.csv & pred_swa.csv
    (Verified: No text columns, Correct folder name)
============================================================
```

[3]:

[3]: